

Reviewer Pack — Talagrand L1 Influence Spread of Clause-Falsification Polyno...

Entry 9e12285a05e1 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 00:46:05 UTC

Talagrand L1 Influence Spread of Clause-Falsification Polynomial Lower-Bounds Tree-Resolution

Entry ID: 9e12285a05e1 Verdict: INCONCLUSIVE Recorded: 2026-05-19 00:46:05 UTC

1. Conjecture

Field A (mathematical branch): Talagrand L1-influence-vector / KKL-Friedgut Boolean Fourier analysis — the Talagrand 1994 functional $T(p) := \sum_i \sqrt{\text{Inf}_i(p)}$, with $\text{Inf}_i(p) = \sum_{S \ni i} \hat{p}(S)^2$, applied as a SYNTACTIC Fourier-analytic invariant of the clause-falsification pseudo-Boolean polynomial $p_F(x) := \#\{C \in F : x \text{ falsifies } C\}$ of a CNF (a property of the clause LIST — multiplicities and polarities — not of the truth table; two unsat 3-CNFs with identical satisfiability function $\equiv \text{FALSE}$ can have very different $T(F)$). Distinct from blacklisted Mansour L1 mass $\sum_S |\hat{p}(S)|$ (a coefficient-indexed absolute-value sum), Bonami 4/2 ratio $\|p\|_4 / \|p\|_2$ (a hypercontractive norm ratio), Bourgain NSM $\sum_S (1 - \rho^{|S|}) \hat{p}(S)^2$ (a level-weighted L^2 mass), hypercontractive term-pair noise log-ratio (a Rényi-2-to-4 kernel statistic), and cross-side Fourier symmetric-difference mass (a truth-table communication-matrix Fourier object that hit NATURAL PROOFS) — Talagrand L1 is the unique coordinate-indexed $\sum_i \sqrt{(\sum_{S \ni i} \hat{p}(S)^2)}$ functional with an OUTER $\sqrt{}$ over per-variable L^2 influence buckets. The functional uses $\sqrt{}$ on integer-valued partial sums and a per-coordinate sum that does NOT extend to a polynomial F_q lift (square-root of an L^2 influence has no polynomial-ring surrogate), Aaronson-Wigderson algebrization-safe; structural shielding from Razborov-Rudich since $T(F)$ is not a poly-time-computable property of the truth table of F (which is $\equiv \text{FALSE}$). ArXiv ‘Talagrand L1 influence sum’ AND (‘DPLL’ OR ‘tree-Resolution’ OR ‘CNF refutation’ OR ‘proof complexity’) returns 0 direct hits and <5 adjacent papers (KKL/Friedgut juntas, never on syntactic clause-falsification polynomials). **Field B** (complexity object): Tree-like Resolution / DPLL refutation size $t(F)$ for unsatisfiable 3-CNFs F in the Ben-Sasson-Wigderson 2001 / Beame-Karp-Pitassi-Saks 2002 / Krajíček 1995 bounded-arithmetic regime where $\neg F$ is Σ^b_0 and $\log_2 t(F)$ lower-bounds V^0 -witnessing complexity; the canonical Frege/Extended-Frege depth stepping stone via Pudlák-Buss feasible interpolation.

Statement:

For every unsatisfiable 3-CNF F on $n \leq 40$ variables with m clauses, let $p_F : \{0,1\}^n \rightarrow \mathbb{Z}_{\geq 0}$ count falsified clauses, expand p_F in the Walsh basis (only subsets S with $|S| \leq 3$ are non-zero), and define $T(F) := \sum_{i=1}^n \sqrt{(\sum_{S \ni i, 1 \leq |S| \leq 3} \hat{p}_F(S)^2)}$. Then $\log_2(1 + t(F)) \geq 0.10 \cdot T(F) / \sqrt{m}$, where $t(F)$ is the DPLL refutation size (backtrack count) under the unit-propagation + first-unassigned variable order. A single F violating this inequality (i.e. $\log_2(1 + t(F)) < 0.10 \cdot T(F) / \sqrt{m}$) refutes the conjecture.

Rationale (proposer’s reasoning):

Clause-falsification influence $\text{Inf}_i(p_F)$ measures how strongly variable i ‘pivots’ the count of broken clauses, and Talagrand’s L^1 -spread functional $\sum_i \sqrt{\text{Inf}_i}$ interpolates between the variance $\sum_i \text{Inf}_i (= I(p_F))$, the total clause-sensitivity) and the maximum influence KKL bound, capturing per-variable contention without collapsing to a coefficient sum (Mansour) or a hypercontractive norm ratio (Bonami). Resolution / DPLL must spend $\geq \Omega(\text{width})$ steps to refute, and Ben-Sasson-Wigderson’s width-size tradeoff together with Talagrand’s variance-spread inequality suggest the clause-list-side $T(F)$ functional should track refutation width, hence $\log t^*$. The bridge is structural (not a truth-table property), uses non-polynomial operations ($\sqrt{\cdot}$, $|\cdot|$), and lies outside both the Razborov-Rudich constructivity regime and the Aaronson-Wigderson algebrization closure.

Taxonomy category: FOURIER_ANALYTIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): bed4a5c9ba1f6b7e

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across all $(n, \alpha) \in \{16, 20, 24, 28, 32\} \times \{4.0, 4.5, 5.0\}$ with 30 seeds each (≤ 450 instances), filter to unsat F . Conjecture is SUPPORTED iff every unsat instance satisfies $\log_2(1+B(F)) \geq 0.10 \cdot T(F)/\sqrt{m}$ AND Pearson r between $\log_2(1+B(F))$ and $T(F)/\sqrt{m}$ over unsat instances is ≥ 0.3 . FALSIFIED if any single unsat F violates the inequality.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL PROOFS	SAFE	0.88	SAFE	SAFE
ALGEBRIZATION	SAFE	0.84	SAFE	SAFE
KARP_LIPTON	SAFE	0.92	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Talagrand influence sum proof complexity Resolution - Fourier analysis CNF refutation DPLL lower bound - Boolean influence tree Resolution clause falsification polynomial

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
```

```
random.seed(seed)
```

```
def generate_cnf(n, m):
```

```
    cnf = []
```

```
    for _ in range(m):
```

```
        clause = []
```

```
        for _ in range(3):
```

```
            var = random.randint(1, n)
```

```
            polarity = random.choice(['x', '¬'])
```

```
            clause.append(f"{polarity}x{var}")
```

```
        cnf.append(" v ".join(clause))
```

```
    return " ^ ".join(cnf)
```

```
def is_unsatisfiable(cnf):
```

```
    n = max(int(literal[2:]) if literal.startswith('x') else int(literal[3:]) for literal in cnf.split(" v "))
```

```
    clauses = cnf.split(" ^ ")
```

```
    variables = set()
```

```
    for clause in clauses:
```

```
        literals = clause.split(" v ")
```

```
        for literal in literals:
```

```
            var = int(literal[2:]) if literal.startswith('x') else int(literal[3:])
```

```
            variables.add(var)
```

```
def dpll(clauses, assignment):
```

```
    if not clauses:
```

```
        return True
```

```
    unit_clause = next((c for c in clauses if len(c.split(" v ")) == 1), None)
```

```
    if unit_clause:
```

```
        literal = unit_clause.strip()
```

```
        polarity = 'x' if literal.startswith('x') else '¬'
```

```
        var = int(literal[2:]) if literal.startswith('x') else int(literal[3:])
```

```
        assignment[var] = polarity == 'x'
```

```
        return dpll([c for c in clauses if not (polarity + f"x{var}" in c or f"¬x{var}" in c)], assignment)
```

```
    pure_literal = next((l for l in variables if all(polarity + f"x{l}" in c or f"¬x{l}" in c for c in clauses)), None)
```

```
    if pure_literal:
```

```
        polarity = 'x' if pure_literal.startswith('x') else '¬'
```

```
        var = int(pure_literal[2:]) if pure_literal.startswith('x') else int(pure_literal[3:])
```

```
        assignment[var] = polarity == 'x'
```

```
        return dpll([c for c in clauses if not (polarity + f"x{var}" in c or f"¬x{var}" in c)], assignment)
```

```
    literal = random.choice(candidates)
```

```
    polarity = 'x' if literal.startswith('x') else '¬'
```

```
    var = int(literal[2:]) if literal.startswith('x') else int(literal[3:])
```

```
    assignment[var] = polarity == 'x'
```

```
    return dpll([c for c in clauses if not (polarity + f"x{var}" in c or f"¬x{var}" in c)], assignment)
```

```
return not dpll(clauses, {})
```

```
def walsh_transform(cnf):
```

```
    n = max(int(literal[2:]) if literal.startswith('x') else int(literal[3:]) for literal in cnf.split(" v "))
```

```
    clauses = cnf.split(" ^ ")
```

```
    p_F = [0] * (1 << n)
```

```
    for clause in clauses:
```

```
        literals = clause.split(" v ")
```

```
        for S in range(1, 1 << n):
```

```
            if all(literal[2:] if literal.startswith('x') else literal[3:] in bin(S)[2:].zfill(n) for literal in literals):
```

```
                p_F[S] += (-1)**sum(int(literal[2:]) if literal.startswith('x') else int(literal[3:]) for literal in literals)
```

```

    return p_F

def dpll_refutation_size(cnf):
    n = max(int(literal[2:]) if literal.startswith('x') else int(literal[3:]) for literal in cnf)
    clauses = cnf.split(" ^ ")
    variables = set()
    for clause in clauses:
        literals = clause.split(" v ")
        for literal in literals:
            var = int(literal[2:]) if literal.startswith('x') else int(literal[3:])
            variables.add(var)

def dpll(clauses, assignment):
    if not clauses:
        return 0
    unit_clause = next((c for c in clauses if len(c.split(" v ")) == 1), None)
    if unit_clause:
        literal = unit_clause.strip()
        polarity = 'x' if literal.startswith('x') else '¬'
        var = int(literal[2:]) if literal.startswith('x') else int(literal[3:])
        assignment[var] = polarity == 'x'
        return 1 + dpll([c for c in clauses if not (polarity + f"x{var}" in c or f"¬x{var}" in c)])
    pure_literal = next((l for l in variables if all(polarity + f"x{l}" in c or f"¬x{l}" in c for c in clauses)), None)
    if pure_literal:
        polarity = 'x' if pure_literal.startswith('x') else '¬'
        var = int(pure_literal[2:]) if pure_literal.startswith('x') else int(pure_literal[3:])
        assignment[var] = polarity == 'x'
        return 1 + dpll([c for c in clauses if not (polarity + f"x{var}" in c or f"¬x{var}" in c)])
    literal = random.choice(clauses.split(" v "))
    polarity = 'x' if literal.startswith('x') else '¬'
    var = int(literal[2:]) if literal.startswith('x') else int(literal[3:])
    assignment[var] = polarity == 'x'
    return 1 + dpll([c for c in clauses if not (polarity + f"x{var}" in c or f"¬x{var}" in c)])

return dpll(clauses, {})

def t_F(cnf):
    p_F = walsh_transform(cnf)
    n = max(int(literal[2:]) if literal.startswith('x') else int(literal[3:]) for literal in cnf)
    return sum(math.sqrt(sum(p_F[S] ** 2 for S in range(1 << n) if bin(S).count('1') == i)) for i in range(1, n+1))

def pearson_correlation(log_2_1_plus_t_F, T_F_over_sqrt_m):
    mean_log = sum(log_2_1_plus_t_F) / len(log_2_1_plus_t_F)
    mean_T = sum(T_F_over_sqrt_m) / len(T_F_over_sqrt_m)
    cov = sum((log_2_1_plus_t_F[i] - mean_log) * (T_F_over_sqrt_m[i] - mean_T) for i in range(len(log_2_1_plus_t_F)))
    var_log = sum((log_2_1_plus_t_F[i] - mean_log) ** 2 for i in range(len(log_2_1_plus_t_F)))
    var_T = sum((T_F_over_sqrt_m[i] - mean_T) ** 2 for i in range(len(T_F_over_sqrt_m))) / len(T_F_over_sqrt_m)
    return cov / (math.sqrt(var_log) * math.sqrt(var_T))

n_values = [16, 20, 24, 28, 32]
alpha_values = [4.0, 4.5, 5.0]
results = []

for n in n_values:
    for alpha in alpha_values:
        m = math.ceil(alpha * n)

```

```

cnf = generate_cnf(n, m)
if not is_unsatisfiable(cnf):
    continue

T_F_val = t_F(cnf)
B_F = dpll_refutation_size(cnf)
log_2_1_plus_t_F = [math.log2(1 + B_F)]
T_F_over_sqrt_m = [T_F_val / math.sqrt(m)]

results.append({
    "n": n,
    "alpha": alpha,
    "m": m,
    "B_F": B_F,
    "log_2_1_plus_t_F": log_2_1_plus_t_F[0],
    "T_F_over_sqrt_m": T_F_over_sqrt_m[0]
})

if not results:
    return {
        "metric_name": "Pearson correlation",
        "metric_value": None,
        "instances_tested": 0,
        "conjecture_holds": False,
        "counterexample": "No unsatisfiable instances found"
    }

log_2_1_plus_t_F = [r["log_2_1_plus_t_F"] for r in results]
T_F_over_sqrt_m = [r["T_F_over_sqrt_m"] for r in results]
correlation = pearson_correlation(log_2_1_plus_t_F, T_F_over_sqrt_m)

return {
    "metric_name": "Pearson correlation",
    "metric_value": correlation,
    "instances_tested": len(results),
    "conjecture_holds": all(math.log2(1 + B_F) >= 0.1 * T_F_val / math.sqrt(m) for r in results),
    "counterexample": "" if all(math.log2(1 + B_F) >= 0.1 * T_F_val / math.sqrt(m) for r in results) else ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(2, 97) for _ in range(30)]

    for seed in seeds:
        trial = run_trial(seed)
        print(f"TRIAL: {trial}")

    results = []
    for n in [16, 20, 24, 28, 32]:
        for alpha in [4.0, 4.5, 5.0]:
            m = math.ceil(alpha * n)
            instances = sum(1 for r in results if r["n"] == n and r["alpha"] == alpha)
            if instances >= 30:
                correlation = pearson_correlation([r["log_2_1_plus_t_F"] for r in results if r["n"] == n and r["alpha"] == alpha])
                support_fraction = sum(1 for r in results if r["n"] == n and r["alpha"] == alpha) / len(results)
                results.append({
                    "n": n,
                    "alpha": alpha,
                    "m": m,
                    "log_2_1_plus_t_F": log_2_1_plus_t_F,
                    "T_F_over_sqrt_m": T_F_over_sqrt_m,
                    "correlation": correlation,
                    "support_fraction": support_fraction
                })

```

```

        "n": n,
        "alpha": alpha,
        "instances_tested": instances,
        "conjecture_holds": support_fraction == 1,
        "correlation": correlation,
        "support_fraction": support_fraction
    })

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={sum(r['correlation'] for r in results) / len(results)} std={sum((r['correlation'] - mean)**2 for r in results) / len(results)}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing = next(r for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\n{n={first_failing['n']}, alpha={first_failing['alpha']}}")
    else:
        print("RESULT: INCONCLUSIVE no unsatisfiable instances found")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_972a393d.py", line 175, in <module>
    trial = run_trial(seed)
            ~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_972a393d.py", line 132, in run_trial
    if not is_unsatisfiable(cnf):
            ~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_972a393d.py", line 33, in is_unsatisfiable
    n = max(int(literal[2:]) if literal.startswith('x') else int(literal[3:]) for literal in cnf.replac
            ~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_972a393d.py", line 33, in <genexpr>
    n = max(int(literal[2:]) if literal.startswith('x') else int(literal[3:]) for literal in cnf.replac
            ~~~~~

```

ValueError: invalid literal for int() with base 10: ''

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a ValueError during CNF parsing (is_unsatisfiable) before any instances were evaluated, so no violation count or Pearson correlation was produced. The pre-registered support condition cannot be assessed. | next: Fix the CNF literal parser in is_unsatisfiable (handle the negation prefix and empty tokens correctly) and rerun the pre-registered sweep over (n,α) with 30 seeds.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	157013
2	preregistration	claude_max	opus	0	0	6043
3	novelty	claude_max	opus	0	0	3963
4	novelty	claude_max	opus	0	0	11873
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16763
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20672
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24947
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	31903
9	judge	claude_max	opus	0	0	6901

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 280077 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 4}

(full prompt+response transcripts available in `research/audit/9e12285a05e1.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/9e12285a05e1.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/9e12285a05e1.tar.gz` (if generated)